

## Protocol 5

[https://github.com/hhafiya/software\\_architecture.git](https://github.com/hhafiya/software_architecture.git), branch `micro_consul`

В цьому завданні я додала до своєї системи **Consul**, який виконує роль `register-service`, `discovery-service` та `config-server`.

1. Всі сервіси самостійно динамічно реєструються при старті Consul, відбувається це в `lifespan` функціях. `Logging-service` має 3 інстанси.

### Services 4 total

| Q Search          | Search Across | Health Status | Service Type | Unhealthy to Healthy |
|-------------------|---------------|---------------|--------------|----------------------|
| ✓ consul          |               |               |              |                      |
| 1 instance        |               |               |              |                      |
| ✓ counter-service |               |               |              |                      |
| 1 instance        |               |               |              |                      |
| ✓ facade-service  |               |               |              |                      |
| 1 instance        |               |               |              |                      |
| ✓ logging-service |               |               |              |                      |
| 3 instances       |               |               |              |                      |

2. Коли `facade-service` звертається до `logging-service` та `counter-service` IP та порти зчитуються з Consul, відповідно статичні адреси не використовуються. В мене це реалізовано у функції `get_service_url`. Крім того я кешую адреси з TTL 5 секунд, бо звернення до Consul кожен запит займає багато часу і так я мінімізую їх кількість, зберігаючи актуальність даних.

```
async def get_service_url(target_service: str):
    now = time.time()
    if target_service in service_cache and service_cache[target_service]["expires_at"] > now:
        return random.choice(service_cache[target_service]["urls"])

    try:
        resp = await app.state.http_client.get(
            f"{CONSUL_URL}/v1/health/service/{target_service}?passing=true", timeout=2.0
        )
        services = resp.json()
        if not services:
            raise HTTPException(status_code=503, detail=f"No healthy nodes for {target_service}")

        urls = [f"http://{s['Service']}:{s['Address']}:{s['Port']}" for s in services]
        service_cache[target_service] = {"urls": urls, "expires_at": now + CACHE_TTL}

        return random.choice(urls)
    except httpx.HTTPError as e:
        if target_service in service_cache:
            return random.choice(service_cache[target_service]["urls"])
        raise HTTPException(status_code=503, detail=f"Consul discovery failed: {e}")
```

3. Налаштування Hazelcast зберігаються як `key/value` Consul і зчитуються `logging-service`. Отримання налаштувань відбувається в `lifespan`. Код звертається до ключа `config/hazelcast/servers` і якщо його знайдено, то список серверів розкодовується з Base54 і використовується для ініціалізації `HazelcastClient`. `facade-service` та `counter-service` роблять

аналогічно, я використовую hazelcast queue, а щоб її отримати треба створити HazelcastClient. Також за ключем config/hazelcast/cluster\_name зберігаю ім'я кластера.

Overview

Services

Nodes

Key/Value

Intentions

ACCESS CONTROLS

Tokens

Policies

## hazelcast

Type ▾ Folders

Name

cluster\_name

servers

< Key / Values / config / hazelcast

## cluster\_name

Value

YAML

1 log

< Key / Values / config / hazelcast

## servers

Value

YAML

1 hz-node-1:5701,hz-node-2:5701,hz-node-3:5701

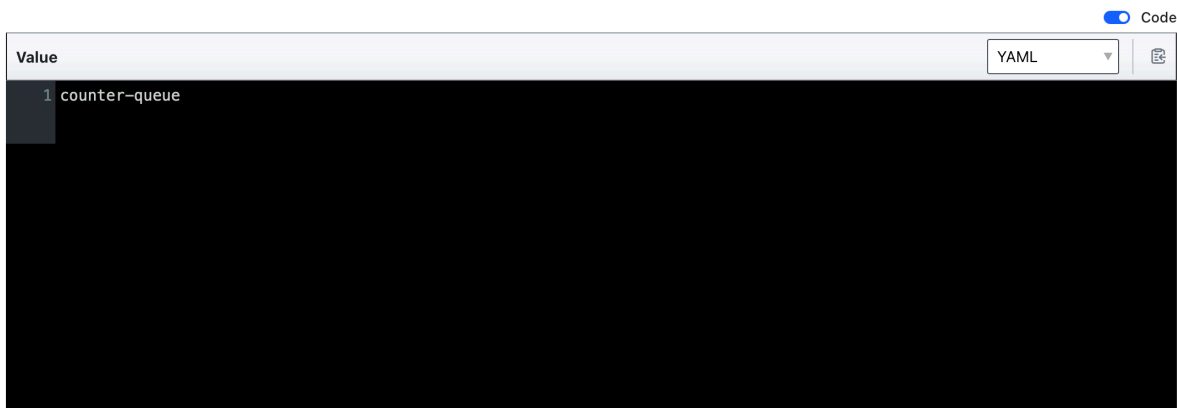
```
# Fetch configurations from Consul KV
hz_servers_str = await fetch_kv(http_client, "config/hazelcast/servers")
hz_servers = hz_servers_str.split(",") if hz_servers_str else ["hz-node-1:5701"]
hz_cluster_name = await fetch_kv(http_client, "config/hazelcast/cluster_name") or "log"
```

Logging

4. Налаштування Message Queue мають зберігатись як key/value у Consul та зчитуватись facade-service та counter-service. У моїй системі достатньо лише ім'я черги за ключем config/mq/queue\_name.

< Key / Values / config / mq

## queue\_name



```
# Read settings from Config Server (Consul KV)
hz_servers_str = await fetch_kv(app_.state.http_client, "config/hazelcast/servers")
hz_servers = hz_servers_str.split(",") if hz_servers_str else ["hz-node-1:5701"]
hz_cluster_name = await fetch_kv(app_.state.http_client, "config/hazelcast/cluster_name") or "log"
queue_name = await fetch_kv(app_.state.http_client, "config/mq/queue_name") or "counter-queue"

hz_client = hazelcast.HazelcastClient(cluster_members=hz_servers, cluster_name=hz_cluster_name)
app_.state.queue = hz_client.get_queue(queue_name).blocking()
```

## Facade

```
hz_servers_str = await fetch_kv(http_client, "config/hazelcast/servers")
hz_servers = hz_servers_str.split(",") if hz_servers_str else ["hz-node-1:5701"]
hz_cluster_name = await fetch_kv(http_client, "config/hazelcast/cluster_name") or "log"
queue_name = await fetch_kv(http_client, "config/mq/queue_name") or "counter-queue"

def start_queue_worker(hz_servers, queue_name, hz_cluster_name):
    hz_client = hazelcast.HazelcastClient(cluster_members=hz_servers, cluster_name=hz_cluster_name)
    queue = hz_client.get_queue(queue_name).blocking()
    db_conn = psycopg2.connect(DATABASE_URL)
```

## Counter

Також, щоб вже не було статичних адрес в коді, я створила окремий ключ ще і для db.

## url

| Value                                         |  | Code |
|-----------------------------------------------|--|------|
| 1 postgresql://user:password@db:5432/balances |  | YAML |

5. Показати, що у випадку відключення екземпляру певного мікросервісу, це буде відображатися у Consul, а виклики будуть переправлятися до інших працюючих екземплярів. Продемонструю це на прикладі logging-service, бо лише він має 3 інстанси. З них я залишила спочатку 2 і зробила транзакцію, а потім залишила лише 1 і зробила транзакцію ще раз. В будь-якому випадку система одразу передала запит до живого інстансу. Це можна помітити на скріншотах з логами фасад сервісу. Для перевірки використала test.py з двома клієнтами та 2 запитами на кожного.

### logging-service

2 instances

## logging-service

| Instances                                                                                                     | Intentions | Routing | Tags |
|---------------------------------------------------------------------------------------------------------------|------------|---------|------|
| <div><div>Q Search</div><div>Search Across</div><div>Health Status</div><div>Unhealthy to Healthy</div></div> |            |         |      |
| logging-service-logging-service-1                                                                             |            |         |      |
| All service checks passing All node checks passing 5e2440258aaf logging-service-1:8000                        |            |         |      |
| logging-service-logging-service-3                                                                             |            |         |      |
| All service checks passing All node checks passing 5e2440258aaf logging-service-3:8000                        |            |         |      |

## logging-service

| Instances                                                                                                     | Intentions | Routing | Tags |
|---------------------------------------------------------------------------------------------------------------|------------|---------|------|
| <div><div>Q Search</div><div>Search Across</div><div>Health Status</div><div>Unhealthy to Healthy</div></div> |            |         |      |
| logging-service-logging-service-3                                                                             |            |         |      |
| All service checks passing All node checks passing 5e2440258aaf logging-service-3:8000                        |            |         |      |

```
DEBUG: Facade is sending log to -> http://logging-service-3:8000
INFO: 151.101.0.223:59408 - "POST /transaction HTTP/1.1" 200 OK
INFO: 151.101.0.223:57622 - "POST /transaction HTTP/1.1" 200 OK
DEBUG: Facade is sending log to -> http://logging-service-1:8000
DEBUG: Facade is sending log to -> http://logging-service-3:8000
INFO: 151.101.0.223:57622 - "POST /transaction HTTP/1.1" 200 OK
INFO: 151.101.0.223:59408 - "POST /transaction HTTP/1.1" 200 OK
INFO: 151.101.0.223:59408 - "GET /stats HTTP/1.1" 200 OK
INFO: 151.101.0.223:59408 - "GET /accounts HTTP/1.1" 200 OK
INFO: 151.101.0.223:59408 - "POST /reset HTTP/1.1" 200 OK
DEBUG: Facade is sending log to -> http://logging-service-2:8000
DEBUG: Facade is sending log to -> http://logging-service-2:8000
INFO: 151.101.0.223:52620 - "POST /transaction HTTP/1.1" 200 OK
INFO: 151.101.0.223:34986 - "POST /transaction HTTP/1.1" 200 OK
DEBUG: Facade is sending log to -> http://logging-service-2:8000
DEBUG: Facade is sending log to -> http://logging-service-2:8000
```

```
DEBUG: Facade is sending log to -> http://logging-service-3:8000
DEBUG: Facade is sending log to -> http://logging-service-3:8000
INFO: 151.101.0.223:19110 - "POST /transaction HTTP/1.1" 200 OK
DEBUG: Facade is sending log to -> http://logging-service-3:8000
INFO: 151.101.0.223:44663 - "POST /transaction HTTP/1.1" 200 OK
INFO: 151.101.0.223:19110 - "POST /transaction HTTP/1.1" 200 OK
DEBUG: Facade is sending log to -> http://logging-service-3:8000
INFO: 151.101.0.223:44663 - "POST /transaction HTTP/1.1" 200 OK
INFO: 151.101.0.223:19110 - "GET /stats HTTP/1.1" 200 OK
INFO: 151.101.0.223:19110 - "GET /accounts HTTP/1.1" 200 OK
INFO: 151.101.0.223:19110 - "POST /reset HTTP/1.1" 200 OK
DEBUG: Facade is sending log to -> http://logging-service-1:8000
DEBUG: Facade is sending log to -> http://logging-service-3:8000
INFO: 151.101.0.223:37314 - "POST /transaction HTTP/1.1" 200 OK
INFO: 151.101.0.223:50311 - "POST /transaction HTTP/1.1" 200 OK
DEBUG: Facade is sending log to -> http://logging-service-3:8000
DEBUG: Facade is sending log to -> http://logging-service-1:8000
```

```
DEBUG: Facade is sending log to -> http://logging-service-3:8000
INFO: 151.101.0.223:60365 - "POST /transaction HTTP/1.1" 200 OK
DEBUG: Facade is sending log to -> http://logging-service-3:8000
INFO: 151.101.0.223:18613 - "POST /transaction HTTP/1.1" 200 OK
INFO: 151.101.0.223:60365 - "POST /transaction HTTP/1.1" 200 OK
DEBUG: Facade is sending log to -> http://logging-service-3:8000
INFO: 151.101.0.223:18613 - "POST /transaction HTTP/1.1" 200 OK
INFO: 151.101.0.223:60365 - "GET /stats HTTP/1.1" 200 OK
INFO: 151.101.0.223:60365 - "GET /accounts HTTP/1.1" 200 OK
INFO: 151.101.0.223:60365 - "POST /reset HTTP/1.1" 200 OK
DEBUG: Facade is sending log to -> http://logging-service-3:8000
DEBUG: Facade is sending log to -> http://logging-service-3:8000
INFO: 151.101.0.223:32985 - "POST /transaction HTTP/1.1" 200 OK
INFO: 151.101.0.223:53329 - "POST /transaction HTTP/1.1" 200 OK
DEBUG: Facade is sending log to -> http://logging-service-3:8000
DEBUG: Facade is sending log to -> http://logging-service-3:8000
```

Коли 1

```
ira@MacBook-Air-Ira software_architecture % docker stop logging-service-2
logging-service-2
ira@MacBook-Air-Ira software_architecture % uv run --with httpx python test.py

Resetting system...
10 clients 10 accounts scenario
Total time: 0.016467499983264133
Requests/sec: 242.90268735783738

Service contribution:
Logging total time: 0.01493316699998104
Counter total time: 0.0062337919989658985
Request count: 4

Final balances:
{'user_0': 2.0, 'user_1': 2.0}

Resetting system...
10 clients 1 account scenario
Total time: 0.01556387500022538
Requests/sec: 257.00540514120524

Service contribution:
Logging total time: 0.015179334001004463
Counter total time: 0.00675237499672221
Request count: 4

Final balances:
{'shared_user': 4.0}

Resetting system...
ira@MacBook-Air-Ira software_architecture % docker stop logging-service-1
logging-service-1
ira@MacBook-Air-Ira software_architecture % uv run --with httpx python test.py

Resetting system...
10 clients 10 accounts scenario
Total time: 0.01805720001348798
Requests/sec: 221.5181880284132

Service contribution:
Logging total time: 0.015961333003360778
Counter total time: 0.005551666999963345
Request count: 4

Final balances:
{'user_0': 2.0, 'user_1': 2.0}

Resetting system...
10 clients 1 account scenario
Total time: 0.015658457996323705
Requests/sec: 255.45299549541343
```

Сервіси саме пропадають з Consul, а не змінюють статус, бо моя реалізація має graceful shutdown.

Ось приклад, якщо прибрати:

```
yield

# Cleanup
# async with httpx.AsyncClient() as http_client:
#     await http_client.put(f"{CONSUL_URL}/v1/agent/service/deregister/{SERVICE_ID}")
# app_.state.hz_client.shutdown()
```

## logging-service

| Instances                                                                                                                                                                                                | Intentions                                 | Routing                                    | Tags                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|--------------------------------------------|---------------------------------------------------|
| <input type="text" value="Search"/>                                                                                                                                                                      | <input type="text" value="Search Across"/> | <input type="text" value="Health Status"/> | <input type="text" value="Unhealthy to Healthy"/> |
| logging-service-logging-service-1                                                                                                                                                                        |                                            |                                            |                                                   |
| <div><div><div></div></div> All service checks failing</div> <div><div><div></div></div> All node checks passing</div> <div><div></div> 5e2440258aaf</div> <div><div></div> logging-service-1:8000</div> |                                            |                                            |                                                   |
| logging-service-logging-service-2                                                                                                                                                                        |                                            |                                            |                                                   |
| <div><div><div></div></div> All service checks failing</div> <div><div><div></div></div> All node checks passing</div> <div><div></div> 5e2440258aaf</div> <div><div></div> logging-service-2:8000</div> |                                            |                                            |                                                   |
| logging-service-logging-service-3                                                                                                                                                                        |                                            |                                            |                                                   |
| <div><div><div></div></div> All service checks passing</div> <div><div><div></div></div> All node checks passing</div> <div><div></div> 5e2440258aaf</div> <div><div></div> logging-service-3:8000</div> |                                            |                                            |                                                   |

## Тестування продуктивності

Використовувала скрипт test.py, що і в завданні 1 та 3.

Результат:

```
Resetting system...
10 clients 10 accounts scenario
Total time: 209.16760362501373
Requests/sec: 478.085507826898

Service contribution:
Logging total time: 1251.2929616498327
Counter total time: 353.6360656971201
Request count: 100000

Final balances:
{'user_8': 10000.0, 'user_3': 10000.0, 'user_7': 10000.0, 'user_1': 10000.0, 'user_2': 10000.0, 'user_5': 10000.0, 'user_0': 10000.0, 'user_4': 10000.0, 'user_6': 10000.0, 'user_9': 10000.0}

Resetting system...
10 clients 1 account scenario
Total time: 230.73709929100005
Requests/sec: 433.3936775112286

Service contribution:
Logging total time: 1389.5426325940043
Counter total time: 388.5125967380686
Request count: 100000

Final balances:
{'shared_user': 100000.0}

Resetting system...
```

| Test scenarios | Task 1 (in-mem)             | Task 3 (DB)                 | Task 5 (final)              |
|----------------|-----------------------------|-----------------------------|-----------------------------|
| 10 accounts    | Total time: 215.49          | Total time: 344.26          | Total time: 209.17          |
|                | logging-service:<br>1272.59 | logging-service:<br>1565.12 | logging-service:<br>1251.29 |
|                | counter-service:<br>1255.08 | counter-service:<br>2423.50 | counter-service:<br>353.64  |
| 1 account      | Total time: 232.75          | Total time: 390.17          | Total time: 230.74          |
|                | logging-service:<br>1371.23 | logging-service:<br>2021.25 | logging-service:<br>1389.54 |
|                | counter-service:<br>1358.61 | counter-service:<br>2771.67 | counter-service:<br>388.51  |

### Counter time:

У першому завданні всі дані зберігалися в оперативній пам'яті, це найшвидший варіант зберігання, тому загальний час виконання (215 сек / 232 сек) є досить малим. Але, ця архітектура не підходить для реальних систем, бо при перезапуску сервісу всі дані втрачаються.

У третьому завданні треба було додати реляційну базу даних (PostgreSQL), щоб надійно зберігати баланси. Для сценарію «10 акаунтів» час counter-service зріс майже вдвічі (з 1255 до 2423 сек). Це сталося через Disk I/O, бо запис на фізичний диск є найповільнішою операцією). facade-сервіс мав чекати, поки база даних збереже дані, перш ніж відповісти клієнту. Для сценарію «1 акаунт» продуктивність впала ще гірше(counter-service час 2771 сек). Всі 100 000 запитів намагалися оновити один і той самий рядок у базі даних одночасно, тому PostgreSQL вибудовував їх у чергу, а facade-сервіс був заблокований, очікуючи зняття блокування бази даних.

І у п'ятому додалися Consul (Service Discovery) та Hazelcast Message Queue. За рахунок черги facade-сервіс миттєво кладе транзакцію в чергу повідомлень, яка знаходиться в RAM) і одразу завершує роботу. Це перетворило процес із синхронного на асинхронний. Зникла проблема блокування, бо facade не чекає на зняття блокувань БД, база даних оновлюється у фоновому режимі окремим воркером. І тому час значно зменшився.

### **Logging time:**

У першому дані зберігалися у звичайному словнику `app.state.history = {}`, а запис у змінну Python відбувається дуже швидко.

У третьому додалося підключення до hazelcast мапи, тобто тоді з'явився мережевий запит до кластера. До того ж, через реплікацію логер мав чекати на відповідь від кластеру про те, що дані успішно скопійовані на усі 3 ноди. Оскільки facade блокувався через базу даних, то навіть паралельні запити до logging-service починали оброблятися із затримкою і час виріс вдвічі. Ще додаю, що тут уже використовувалося 3 екземпляри logging-service, між якими facade самостійно розподіляв запити, що підвищило доступність системи.

І в п'ятому додався Consul, але час при цьому зменшився порівняно з третім і став приблизно на рівні першого. Причини є дві: 1. facade не запитує Consul при кожному запиті. Він один раз дізнається IP-адресу logging-сервісу, зберігає її в пам'яті на 5 секунд і сотні разів використовує напрямку. Мережеві витрати на service discovery максимально зменшені. 2. Оскільки counter-service тепер використовує message queue, facade більше не гальмує і максимально швидко, паралельно відправляє HTTP-запити до logging-service через `asyncio.gather`.

### **Висновок**

Найгірший загальний час мала архітектура з прямим підключенням до бази даних без використання черг (третє завдання). Загальний час виконання зріс до 344.26 сек для 10 акаунтів та до 390.17 сек для одного акаунта. Причина - операції Disk I/O та синхронне очікування відповіді від БД.

Додавання асинхронної обробки через Hazelcast Message Queue у п'ятому дозволило досягти найкращого результату - 209.17 сек для 10 акаунтів, що навіть швидше за початковий In-memoгу варіант (215.49 сек).

В результаті, фінальна архітектура значно більш надійна за першу(БД та черга) та масштабована(Consul), одночасно вона найбільш продуктивна за рахунок асинхронності.